

# Advanced gRPC

Why my client / server is not working ?

# Table of Content

- Show case
- Debugging
- Load balancing
- Pooling connections
- Error analysis

# Show case

Why my client / server not working ?

# Show case

```
panic: intended panic ! boom ~ [recovered]
      panic: intended panic ! boom ~
```

**Server**

- Deliberately break the server

```
rpc error: code = Unavailable desc = error reading from server: EOF
```

**Client**

# TCPDUMP

- The hard way

```
$ tcpdump -i lo0 -w panic.pcap
tcpdump: listening on lo0, link-type NULL (BSD loopback), capture size 262144 bytes
104 packets captured
110 packets received by filter
0 packets dropped by kernel
```

| No. | Protocol | Info                                                                                 | Source Port | Destination Port |
|-----|----------|--------------------------------------------------------------------------------------|-------------|------------------|
| 41  | TCP      | 56798 → 8023 [SYN] Seq=0 Win=65535 Len=0 MSS=16344 WS=64 TSval=1851788431 TSecr=0... | 56798       | 8023             |
| 42  | TCP      | 8023 → 56798 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=16344 WS=64 TSval=7265224... | 8023        | 56798            |
| 43  | TCP      | 56798 → 8023 [ACK] Seq=1 Ack=1 Win=408256 Len=0 TSval=1851788431 TSecr=726522475     | 56798       | 8023             |
| 44  | TCP      | [TCP Window Update] 8023 → 56798 [ACK] Seq=1 Ack=1 Win=408256 Len=0 TSval=7265224... | 8023        | 56798            |
| 45  | HTTP2    | Magic                                                                                | 56798       | 8023             |
| 46  | HTTP2    | SETTINGS[0]                                                                          | 56798       | 8023             |
| 47  | TCP      | 8023 → 56798 [ACK] Seq=1 Ack=25 Win=408256 Len=0 TSval=726522475 TSecr=1851788431    | 8023        | 56798            |
| 48  | TCP      | 8023 → 56798 [ACK] Seq=1 Ack=34 Win=408256 Len=0 TSval=726522475 TSecr=1851788431    | 8023        | 56798            |
| 49  | HTTP2    | SETTINGS[0]                                                                          | 8023        | 56798            |
| 50  | TCP      | 56798 → 8023 [ACK] Seq=34 Ack=16 Win=408256 Len=0 TSval=1851788431 TSecr=726522475   | 56798       | 8023             |
| 51  | HTTP2    | SETTINGS[0]                                                                          | 56798       | 8023             |
| 52  | TCP      | 8023 → 56798 [ACK] Seq=16 Ack=43 Win=408256 Len=0 TSval=726522475 TSecr=1851788431   | 8023        | 56798            |
| 53  | HTTP2    | SETTINGS[0]                                                                          | 8023        | 56798            |
| 54  | TCP      | 56798 → 8023 [ACK] Seq=43 Ack=25 Win=408256 Len=0 TSval=1851788431 TSecr=726522475   | 56798       | 8023             |
| 55  | HTTP2    | HEADERS[1]: POST /example.Example/Unary                                              | 56798       | 8023             |
| 56  | TCP      | 8023 → 56798 [ACK] Seq=25 Ack=139 Win=408128 Len=0 TSval=726522475 TSecr=18517884... | 8023        | 56798            |
| 57  | GRPC     | DATA[1] (GRPC) (PROTOBUF)                                                            | 56798       | 8023             |
| 58  | TCP      | 8023 → 56798 [ACK] Seq=25 Ack=169 Win=408128 Len=0 TSval=726522476 TSecr=18517884... | 8023        | 56798            |
| 59  | HTTP2    | WINDOW_UPDATE[0], PING[0]                                                            | 8023        | 56798            |
| 60  | TCP      | 56798 → 8023 [ACK] Seq=169 Ack=55 Win=408192 Len=0 TSval=1851788432 TSecr=7265224... | 56798       | 8023             |
| 61  | HTTP2    | PING[0]                                                                              | 56798       | 8023             |
| 62  | TCP      | 8023 → 56798 [ACK] Seq=55 Ack=186 Win=408064 Len=0 TSval=726522476 TSecr=18517884... | 8023        | 56798            |
| 63  | HTTP2    | HEADERS[1]: 200 OK, DATA[1]                                                          | 8023        | 56798            |
| 64  | GRPC     | HEADERS[1] (GRPC) (PROTOBUF)                                                         | 8023        | 56798            |
| 65  | TCP      | 56798 → 8023 [ACK] Seq=186 Ack=108 Win=408192 Len=0 TSval=1851788434 TSecr=726522... | 56798       | 8023             |
| 66  | TCP      | 56798 → 8023 [ACK] Seq=186 Ack=141 Win=408128 Len=0 TSval=1851788434 TSecr=726522... | 56798       | 8023             |
| 67  | HTTP2    | WINDOW_UPDATE[0], PING[0]                                                            | 56798       | 8023             |
| 68  | TCP      | 8023 → 56798 [ACK] Seq=141 Ack=216 Win=408064 Len=0 TSval=726522478 TSecr=1851788... | 8023        | 56798            |
| 69  | HTTP2    | PING[0]                                                                              | 8023        | 56798            |
| 70  | TCP      | 56798 → 8023 [ACK] Seq=216 Ack=158 Win=408128 Len=0 TSval=1851788434 TSecr=726522... | 56798       | 8023             |
| 71  | TCP      | 56798 → 8023 [FIN, ACK] Seq=216 Ack=158 Win=408128 Len=0 TSval=1851788435 TSecr=7... | 56798       | 8023             |
| 72  | TCP      | 8023 → 56798 [ACK] Seq=158 Ack=217 Win=408064 Len=0 TSval=726522479 TSecr=1851788... | 8023        | 56798            |
| 73  | TCP      | 8023 → 56798 [FIN, ACK] Seq=158 Ack=217 Win=408064 Len=0 TSval=726522479 TSecr=18... | 8023        | 56798            |
| 74  | TCP      | 56798 → 8023 [ACK] Seq=217 Ack=159 Win=408128 Len=0 TSval=1851788435 TSecr=726522... | 56798       | 8023             |

Establish connection

Request

Response

Close connection

Compares to success  
normal-unary.pcap

| tcp.port == 8023 |          |                                                                                      |             |                  |  |
|------------------|----------|--------------------------------------------------------------------------------------|-------------|------------------|--|
| No.              | Protocol | Info                                                                                 | Source Port | Destination Port |  |
| 56               | TCP      | 56836 → 8023 [SYN] Seq=0 Win=65535 Len=0 MSS=16344 WS=64 TSval=1004075305 TSecr=0... | 56836       | 8023             |  |
| 57               | TCP      | 8023 → 56836 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=16344 WS=64 TSval=3379225... | 8023        | 56836            |  |
| 58               | TCP      | 56836 → 8023 [ACK] Seq=1 Ack=1 Win=408256 Len=0 TSval=1004075305 TSecr=3379225361    | 56836       | 8023             |  |
| 59               | TCP      | [TCP Window Update] 8023 → 56836 [ACK] Seq=1 Ack=1 Win=408256 Len=0 TSval=3379225... | 8023        | 56836            |  |
| 60               | HTTP2    | Magic                                                                                | 56836       | 8023             |  |
| 61               | TCP      | 8023 → 56836 [ACK] Seq=1 Ack=25 Win=408256 Len=0 TSval=3379225361 TSecr=1004075305   | 8023        | 56836            |  |
| 62               | HTTP2    | SETTINGS[0]                                                                          | 56836       | 8023             |  |
| 63               | TCP      | 8023 → 56836 [ACK] Seq=1 Ack=34 Win=408256 Len=0 TSval=3379225361 TSecr=1004075305   | 8023        | 56836            |  |
| 64               | HTTP2    | SETTINGS[0]                                                                          | 8023        | 56836            |  |
| 65               | TCP      | 56836 → 8023 [ACK] Seq=34 Ack=16 Win=408256 Len=0 TSval=1004075305 TSecr=33792253... | 56836       | 8023             |  |
| 66               | HTTP2    | SETTINGS[0]                                                                          | 56836       | 8023             |  |
| 67               | TCP      | 8023 → 56836 [ACK] Seq=16 Ack=43 Win=408256 Len=0 TSval=3379225361 TSecr=10040753... | 8023        | 56836            |  |
| 68               | HTTP2    | HEADERS[1]: POST /example.Example/Unary                                              | 56836       | 8023             |  |
| 69               | TCP      | 8023 → 56836 [ACK] Seq=16 Ack=153 Win=408128 Len=0 TSval=3379225361 TSecr=1004075... | 8023        | 56836            |  |
| 70               | HTTP2    | SETTINGS[0]                                                                          | 8023        | 56836            |  |
| 71               | TCP      | 56836 → 8023 [ACK] Seq=153 Ack=25 Win=408256 Len=0 TSval=1004075305 TSecr=3379225... | 56836       | 8023             |  |
| 72               | GRPC     | DATA[1] (GRPC) (PROTOBUF)                                                            | 56836       | 8023             |  |
| 73               | TCP      | 8023 → 56836 [ACK] Seq=25 Ack=183 Win=408064 Len=0 TSval=3379225362 TSecr=1004075... | 8023        | 56836            |  |
| 74               | HTTP2    | WINDOW_UPDATE[0], PING[0]                                                            | 8023        | 56836            |  |
| 75               | TCP      | 56836 → 8023 [ACK] Seq=183 Ack=55 Win=408192 Len=0 TSval=1004075306 TSecr=3379225... | 56836       | 8023             |  |
| 76               | HTTP2    | PING[0]                                                                              | 56836       | 8023             |  |
| 77               | TCP      | 8023 → 56836 [ACK] Seq=55 Ack=200 Win=408064 Len=0 TSval=3379225362 TSecr=1004075... | 8023        | 56836            |  |
| 78               | TCP      | 8023 → 56836 [FIN, ACK] Seq=55 Ack=200 Win=408064 Len=0 TSval=3379225366 TSecr=10... | 8023        | 56836            |  |
| 79               | TCP      | 56836 → 8023 [ACK] Seq=200 Ack=56 Win=408192 Len=0 TSval=1004075310 TSecr=3379225... | 56836       | 8023             |  |
| 80               | TCP      | 56836 → 8023 [FIN, ACK] Seq=200 Ack=56 Win=408192 Len=0 TSval=1004075310 TSecr=33... | 56836       | 8023             |  |
| 81               | TCP      | 8023 → 56836 [ACK] Seq=56 Ack=201 Win=408064 Len=0 TSval=3379225368 TSecr=1004075... | 8023        | 56836            |  |

Establish connection

Request

Close connection

Analyze the crash  
panic.pcap

# Environment variables

- Easier, but not much
- `GRPC_GO_LOG_VERBOSITY_LEVEL=99`  
`GRPC_GO_LOG_SEVERITY_LEVEL=info`
- ~~`GRPC_TRACE=all`~~  
~~`GRPC_VERBOSITY=debug`~~

Create subchannel

Ready

Shutdown

```
2022/12/29 15:34:42 INFO: [core] [Channel #1] Channel created
2022/12/29 15:34:42 INFO: [core] [Channel #1] original dial target is: "127.0.0.1:8023"
2022/12/29 15:34:42 INFO: [core] [Channel #1] dial target "127.0.0.1:8023" parse failed: parse "127.0.0.1:8023": first path segment in URL cannot contain colon
2022/12/29 15:34:42 INFO: [core] [Channel #1] fallback to scheme "passthrough"
2022/12/29 15:34:42 INFO: [core] [Channel #1] parsed dial target is: {Scheme:passthrough Authority: Endpoint:127.0.0.1:8023 URL:{Scheme:passthrough Opaque: Us
2022/12/29 15:34:42 INFO: [core] [Channel #1] Channel authority set to "127.0.0.1:8023"
2022/12/29 15:34:42 INFO: [core] [Channel #1] Resolver state updated: {
  "Addresses": [
    {
      "Addr": "127.0.0.1:8023",
      "ServerName": "",
      "Attributes": null,
      "BalancerAttributes": null,
      "Type": 0,
      "Metadata": null
    }
  ],
  "ServiceConfig": null,
  "Attributes": null
} (resolver returned new addresses)
2022/12/29 15:34:42 INFO: [core] [Channel #1] Channel switches to new LB policy "pick_first"
2022/12/29 15:34:42 INFO: [core] [Channel #1 SubChannel #2] Subchannel created
2022/12/29 15:34:42 INFO: [core] [Channel #1 SubChannel #2] Subchannel Connectivity change to CONNECTING
2022/12/29 15:34:42 INFO: [core] [Channel #1 SubChannel #2] Subchannel picks a new address "127.0.0.1:8023" to connect
2022/12/29 15:34:42 INFO: [core] blockingPicker: the picked transport is not ready, loop back to repick
2022/12/29 15:34:42 INFO: [core] pickfirstBalancer: UpdateSubConnState: 0x140000c4720, {CONNECTING <nil>}
2022/12/29 15:34:42 INFO: [core] [Channel #1] Channel Connectivity change to CONNECTING
2022/12/29 15:34:42 INFO: [core] [Channel #1 SubChannel #2] Subchannel Connectivity change to READY
2022/12/29 15:34:42 INFO: [core] pickfirstBalancer: UpdateSubConnState: 0x140000c4720, {READY <nil>}
2022/12/29 15:34:42 INFO: [core] [Channel #1] Channel Connectivity change to READY
{"level":"debug","ts":1672299282826.489,"caller":"log/log.go:136","msg":"","app":"client","at":1672299282824,"method":"/example.Example/Unary","timeCost":2,"
2022/12/29 15:34:42 request 1 returns response: advanced grpc!
2022/12/29 15:34:42 INFO: [core] [Channel #1] Channel Connectivity change to SHUTDOWN
2022/12/29 15:34:42 INFO: [core] [Channel #1 SubChannel #2] Subchannel Connectivity change to SHUTDOWN
2022/12/29 15:34:42 INFO: [core] [Channel #1 SubChannel #2] Subchannel deleted
2022/12/29 15:34:42 INFO: [core] [Channel #1] Channel deleted
```

# Analyze the logs

## Compares to success

Create connection

Ready -> IDLE

Closing

```

2022/12/29 15:39:08 INFO: [core] [Channel #1] Channel created
2022/12/29 15:39:08 INFO: [core] [Channel #1] original dial target is: "127.0.0.1:8023"
2022/12/29 15:39:08 INFO: [core] [Channel #1] dial target "127.0.0.1:8023" parse failed: parse "127.0.0.1:8023": first path segment in URL cannot contain colons
2022/12/29 15:39:08 INFO: [core] [Channel #1] fallback to scheme "passthrough"
2022/12/29 15:39:08 INFO: [core] [Channel #1] parsed dial target is: {Scheme:passthrough Authority: Endpoint:127.0.0.1:8023 URL:{Scheme:passthrough Opaque:URL:127.0.0.1:8023}}
2022/12/29 15:39:08 INFO: [core] [Channel #1] Channel authority set to "127.0.0.1:8023"
2022/12/29 15:39:08 INFO: [core] [Channel #1] Resolver state updated: {
  "Addresses": [
    {
      "Addr": "127.0.0.1:8023",
      "ServerName": "",
      "Attributes": null,
      "BalancerAttributes": null,
      "Type": 0,
      "Metadata": null
    }
  ],
  "ServiceConfig": null,
  "Attributes": null
} (resolver returned new addresses)
2022/12/29 15:39:08 INFO: [core] [Channel #1] Channel switches to new LB policy "pick_first"
2022/12/29 15:39:08 INFO: [core] [Channel #1 SubChannel #2] Subchannel created
2022/12/29 15:39:08 INFO: [core] [Channel #1 SubChannel #2] Subchannel Connectivity change to CONNECTING
2022/12/29 15:39:08 INFO: [core] [Channel #1 SubChannel #2] Subchannel picks a new address "127.0.0.1:8023" to connect
2022/12/29 15:39:08 INFO: [core] blockingPicker: the picked transport is not ready, loop back to repick
2022/12/29 15:39:08 INFO: [core] pickfirstBalancer: UpdateSubConnState: 0x1400000f3f8, {CONNECTING <nil>}
2022/12/29 15:39:08 INFO: [core] [Channel #1] Channel Connectivity change to CONNECTING
2022/12/29 15:39:08 INFO: [core] [Channel #1 SubChannel #2] Subchannel Connectivity change to READY
2022/12/29 15:39:08 INFO: [core] pickfirstBalancer: UpdateSubConnState: 0x1400000f3f8, {READY <nil>}
2022/12/29 15:39:08 INFO: [core] [Channel #1] Channel Connectivity change to READY
2022/12/29 15:39:08 INFO: [core] [Channel #1 SubChannel #2] Subchannel Connectivity change to IDLE
2022/12/29 15:39:08 INFO: [core] pickfirstBalancer: UpdateSubConnState: 0x1400000f3f8, {IDLE <nil>}
2022/12/29 15:39:08 INFO: [core] [Channel #1] Channel Connectivity change to IDLE
2022/12/29 15:39:08 INFO: [transport] transport: loopyWriter.run returning. connection error: desc = "transport is closing"
{"level":"debug","ts":1672299548372.673,"caller":"log/log.go:136","msg":"","app":"client","at":1672299548363,"method":"/example.Example/Unary","timeCost":9,""}
Error: make grpc unary request 1 failed: rpc error: code = Unavailable desc = error reading from server: EOF

```

# Analyze the logs

## Crashed one

# Channelz

- Runtime info about connections
- Debug live programs

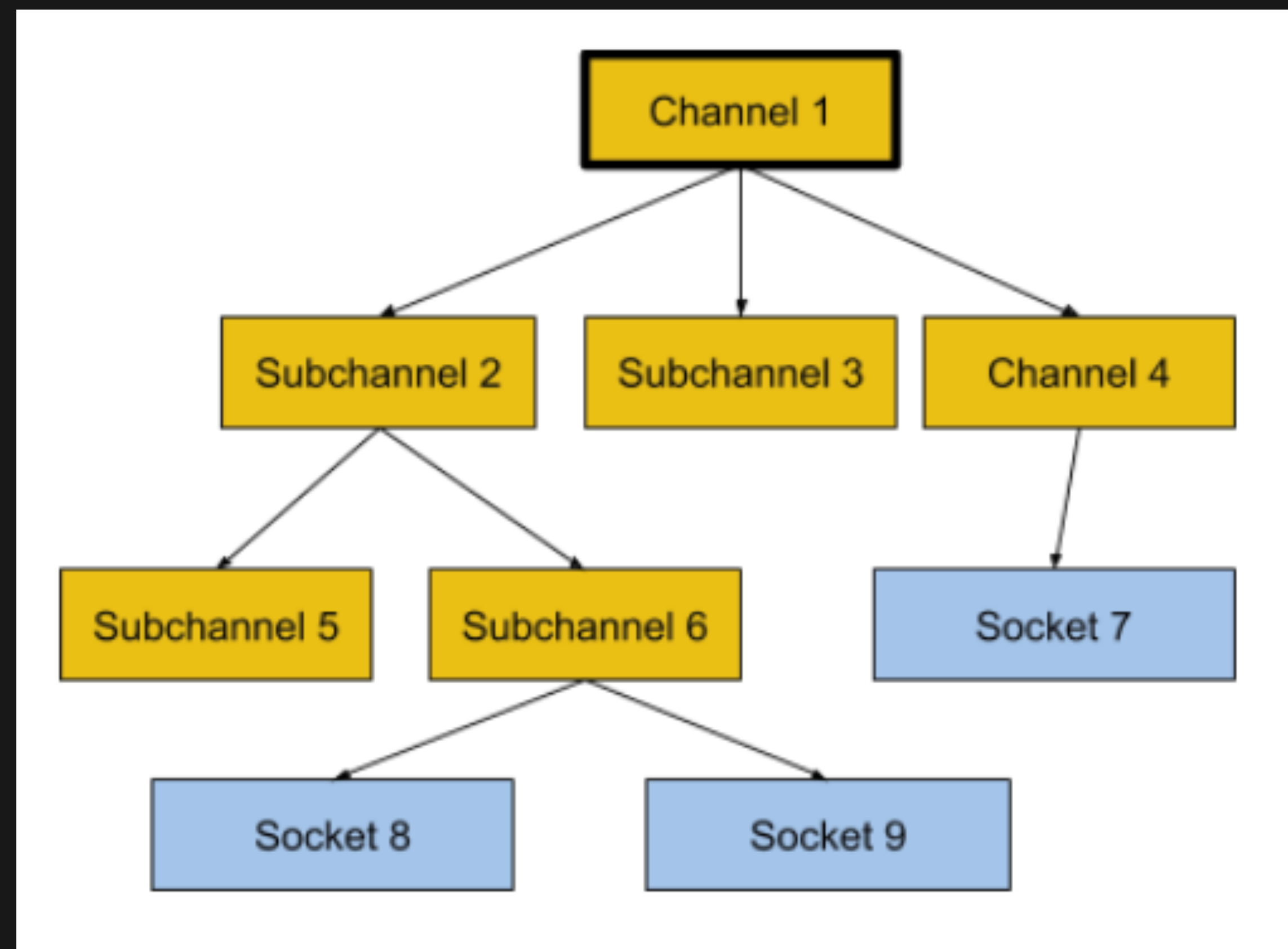
```
import "google.golang.org/grpc/channelz/service"

// s is a *grpc.Server
service.RegisterChannelzServiceToServer(s)

// call s.Serve() to serve channelz service
```

# Channelz Proposal

- Channel:
  - an abstraction that can start and complete RPCs
- Subchannel:
  - an abstraction that is load balanced over by an owning channel
  - sockets
- Socket:
  - file descriptor
  - connection
- Doc: <https://github.com/grpc/proposal/blob/master/A14-channelz.md>



```
channelzcli -k --addr localhost:8023 list server
```

| ID | Name   | LocalAddr | Calls | Success | Fail | LastCall |
|----|--------|-----------|-------|---------|------|----------|
| 1  | <none> | :::8023   | 10    | 9       | 0    | 0ms      |

```
channelzcli -k --addr localhost:8023 describe server 1
```

```
ID:      1
```

```
Name:
```

```
Calls:
```

```
    Started:      12
```

```
    Succeeded:    11
```

```
    Failed:       0
```

```
    LastCallStarted: 2022-12-30 05:03:27.565031 +0000 UTC
```

# Channelz

## Server side



| ID | Name            | State      | Channel | SubChannel | Calls | Success | Fail | LastCall |
|----|-----------------|------------|---------|------------|-------|---------|------|----------|
| 1  | 127.0.0.1:8023  | CONNECTING | 0       | 1          | 0     | 0       | 0    | none     |
| 4  | 127.0.0.1:63694 | READY      | 0       | 1          | 1     | 0       | 0    | 0ms      |

```
ID: 1
Name: 127.0.0.1:8023
State: CONNECTING
Target: 127.0.0.1:8023
Calls:
  Started: 0
  Succeeded: 0
  Failed: 0
  LastCallStarted: none
Socket: <none>
Channels: <none>
Subchannels:
  ID Name State Start Succeeded Failed
  2
Trace:
NumEvents: 10
CreationTimestamp: 2022-12-30 05:01:47.114402 +0000 UTC
Events
Severity Description
INFO Channel created
INFO original dial target is: "127.0.0.1:8023"
INFO dial target "127.0.0.1:8023" parse failed: parse "127.0.0.1:8023": first path segment in URL cannot contain colon
INFO fallback to scheme "passthrough"
INFO parsed dial target is: {Scheme:passthrough Authority: Endpoint:127.0.0.1:8023 URL:{Scheme:passthrough Opaque: User: Host: Path:/127.0.0.1:8023}}
INFO Channel authority set to "127.0.0.1:8023"
INFO Resolver state updated: {
"Addresses": [
  {
    "Addr": "127.0.0.1:8023",
    "ServerName": "",
    "Attributes": null,
    "BalancerAttributes": null,
    "Type": 0,
    "Metadata": null
  }
],
"ServiceConfig": null,
"Attributes": null
} (resolver returned new addresses) 2022-12-30 05:01:47.114594 +0000 UTC
INFO Channel switches to new LB policy "pick_first" 2022-12-30 05:01:47.114603 +0000 UTC
INFO Subchannel(id:2) created 2022-12-30 05:01:47.114613 +0000 UTC
INFO Channel Connectivity change to CONNECTING 2022-12-30 05:01:47.114797 +0000 UTC
```

# Channelz

## Client side



```
ID: 1
Name: 127.0.0.1:8023
State: CONNECTING
Target: 127.0.0.1:8023
Calls:
  Started: 0
  Succeeded: 0
  Failed: 0
  LastCallStarted: none
Socket: <none>
Channels: <none>
Subchannels:
  ID Name State Start Succeeded Failed
  2 2 CONNECTING 0 0 0
Trace:
  NumEvents: 10
  CreationTimestamp: 2022-12-30 05:01:47.114402 +0000 UTC
  Events
    Severity Description
    INFO Channel created
    INFO original dial target is: "127.0.0.1:8023"
    INFO dial target "127.0.0.1:8023" parse failed: parse "127.0.0.1:8023": first path segment in URL cannot contain colon
    INFO fallback to scheme "passthrough"
    INFO parsed dial target is: {Scheme:passthrough Authority: Endpoint:127.0.0.1:8023 URL:{Scheme:passthrough Opaque: User
    INFO Channel authority set to "127.0.0.1:8023"
    INFO Resolver state updated: {
  "Addresses": [
    {
      "Addr": "127.0.0.1:8023",
      "ServerName": "",
      "Attributes": null,
      "BalancerAttributes": null,
      "Type": 0,
      "Metadata": null
    }
  ],
  "ServiceConfig": null,
  "Attributes": null
} (resolver returned new addresses) 2022-12-30 05:01:47.114594 +0000 UTC
  INFO Channel switches to new LB policy "pick_first" 2022-12-30 05:01:47.114603 +0000 UTC
  INFO Subchannel(id:2) created 2022-12-30 05:01:47.114613 +0000 UTC
  INFO Channel Connectivity change to CONNECTING 2022-12-30 05:01:47.114797 +0000 UTC
```

# Channelz

## Client side

# Channelz tool

- Remote pod / service
- <https://gitlab.planetmeican.com/titan/channelz>

```
channelzcli \
  --addr channelz.sandbox.planetmeican.com:443 \
  --target-addr 172.24.160.113:8023 \
  list server
```

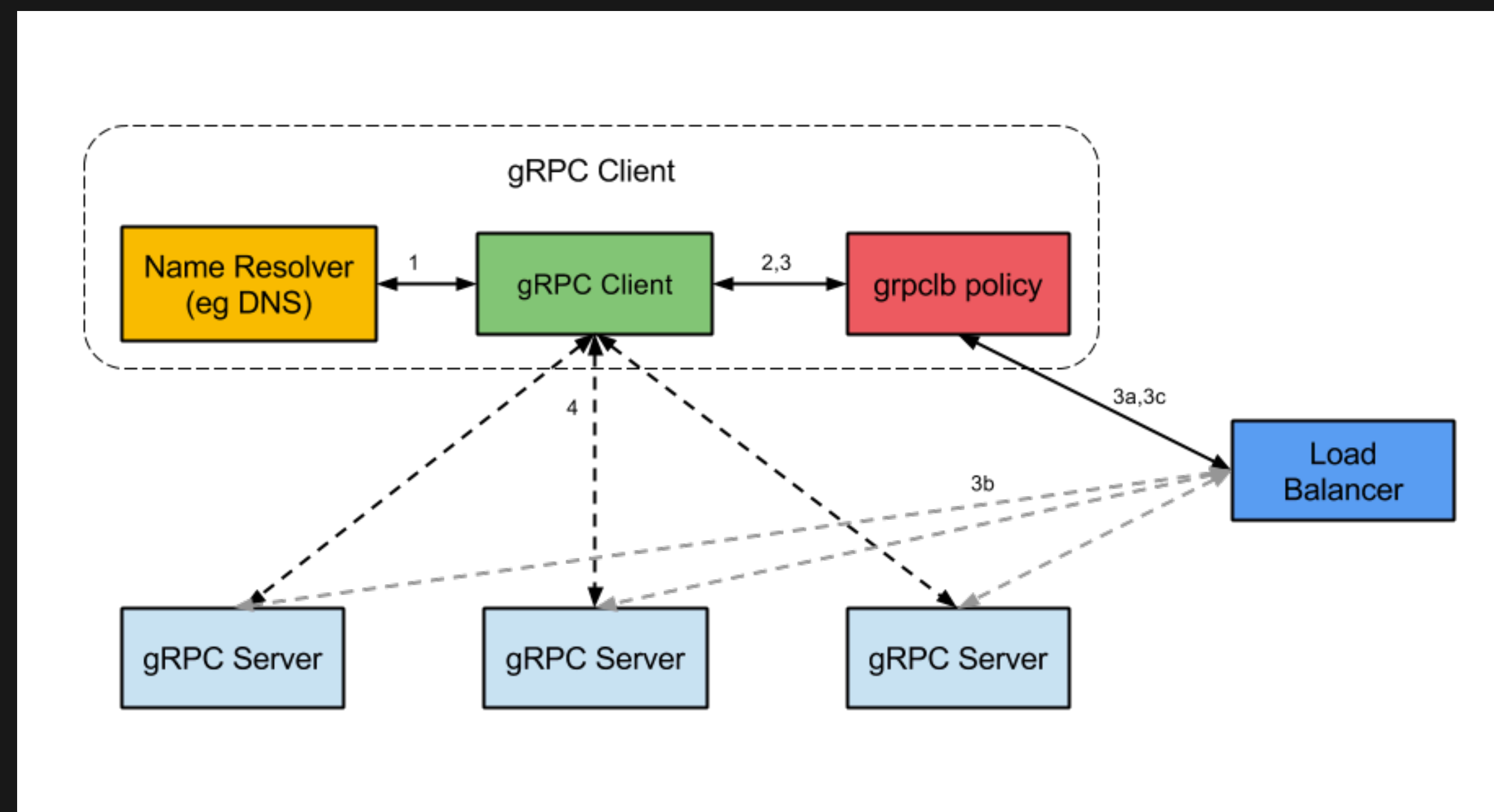
**Pod IP**

```
channelzcli \
  --addr channelz.sandbox.planetmeican.com:443 \
  --target-addr advanced-grpc-server.app-nerds.svc.cluster.local:8023 \
  list server
```

**Service**

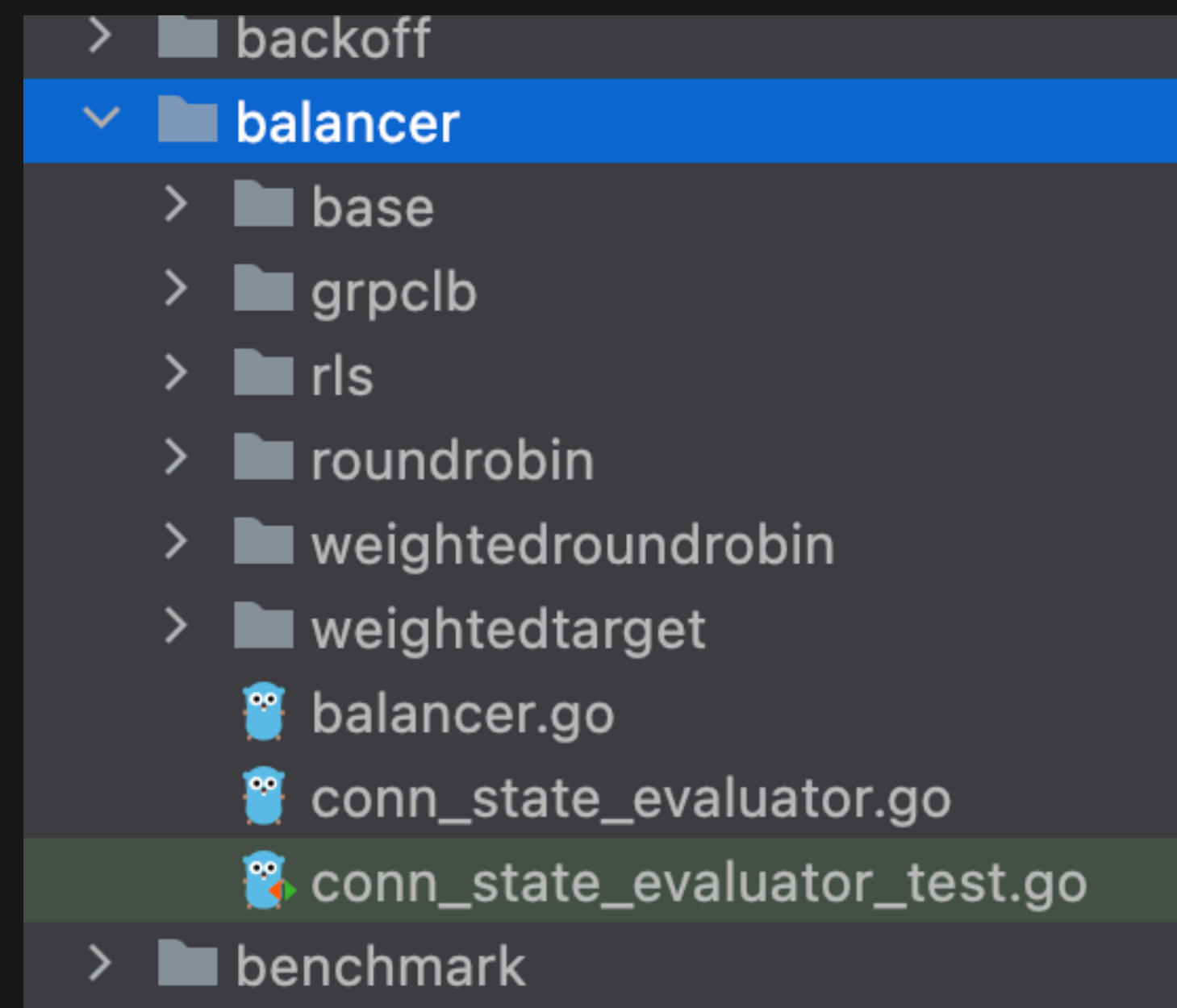
# Load Balancing

- Load balance per call, not per connection
- Balancers:
  - pick\_first
  - round\_robin
  - grpclb
  - xDS
  - etc.



# LB Policy

- Policies:
  - base (used to write your own)
  - ~~grpc~~ `grpc`
  - rls (randomized local search)
  - roundrobin
  - weightedroundrobin
  - weightedtarget
- Code: [google.golang.org/grpc/balancer](https://google.golang.org/grpc/balancer)



# LB Policy

## Definition (protobuf)

```
PickFirstConfig pick_first = 4 [json_name = "pick_first"];

RoundRobinConfig round_robin = 1 [json_name = "round_robin"];

// gRPC lookaside load balancing.
// This will eventually be deprecated by the new xDS-based local
// balancing policy.
GrpcLbConfig grpc_lb = 3;

// REMAINING POLICIES ARE EXPERIMENTAL -- DO NOT USE

PriorityLoadBalancingPolicyConfig priority_experimental = 9
    [json_name = "priority_experimental"];
WeightedTargetLoadBalancingPolicyConfig weighted_target_experimental = 10
    [json_name = "weighted_target_experimental"];
OutlierDetectionLoadBalancingConfig outlier_detection = 15
    [json_name = "outlier_detection_experimental"];
RlsLoadBalancingPolicyConfig rls = 19 [json_name = "rls_experimental"];

// xDS-based load balancing.
XdsClusterManagerLoadBalancingPolicyConfig xds_cluster_manager_experimental
    = 14 [json_name = "xds_cluster_manager_experimental"];
CdsConfig cds_experimental = 6 [json_name = "cds_experimental"];
XdsClusterResolverLoadBalancingPolicyConfig
    xds_cluster_resolver_experimental = 11
    [json_name = "xds_cluster_resolver_experimental"];
XdsClusterImplLoadBalancingPolicyConfig xds_cluster_impl_experimental = 12
    [json_name = "xds_cluster_impl_experimental"];
OverrideHostLoadBalancingPolicyConfig override_host_experimental = 18
    [json_name = "override_host_experimental"];
XdsWrrLocalityLoadBalancingPolicyConfig xds_wrr_locality_experimental = 16
    [json_name = "xds_wrr_locality_experimental"];
RingHashLoadBalancingConfig ring_hash_experimental = 13
    [json_name = "ring_hash_experimental"];
LeastRequestLocalityLoadBalancingPolicyConfig least_request_experimental =
    17 [json_name = "least_request_experimental"];

// Deprecated xDS-related policies.
LrsLoadBalancingPolicyConfig lrs_experimental = 8
    [json_name = "lrs_experimental", deprecated = true];
EdsLoadBalancingPolicyConfig eds_experimental = 7
    [json_name = "eds_experimental", deprecated = true];
XdsConfig xds = 2 [deprecated = true];
XdsConfig xds_experimental = 5 [json_name = "xds_experimental",
    deprecated = true];
```

# LB Policy Usage

```
grpc.WithDefaultServiceConfig(
```

**grpc.DialOption**

```
{
  "loadBalancingPolicy": "round_robin"
}
```

**round\_robin**

```
{
  "loadBalancingPolicy": "weighted_target_experimental",
  "weighted_target_experimental": {
    "targets": {
      "cluster_1": {
        "weight": 75,
        "childPolicy": [
          {
            "grpclb": {
              "childPolicy": [
                {
                  "pick_first": {}
                }
              ]
            }
          ],
          "targetName": "foo-service"
        ]
      },
      "cluster_2": {
        "weight": 25,
        "childPolicy": [
          {
            "round_robin": ""
          }
        ]
      }
    ]
  }
}
```

**weighted\_target\_experimental**

# Pick first

- Default
- One channel
- Use the first subchannel

```
2022-12-30 05:01:47.114594 +0000 UTC
Channel switches to new LB policy "pick_first"
Subchannel(id:2) created
Channel Connectivity change to CONNECTING
```

# Round robin

```
grpc.Dial(endpoint, grpc.WithDefaultServiceConfig(s: `{"loadBalancingPolicy":"round_robin"}`))
```

Add service config

```
2022-12-30 05:26:38.200972 +0000 UTC
Channel switches to new LB policy "round_robin"
Subchannel(id:2) created
Channel Connectivity change to CONNECTING
```

Proof

# Round robin

- Balance between subchannels

```
grpc.Dial(endpoint, grpc.WithDefaultServiceConfig(s: `{"loadBalancingPolicy":"round_robin"}`))
```

Add service config

```
2022-12-30 05:26:38.200972 +0000 UTC
Channel switches to new LB policy "round_robin"
Subchannel(id:2) created
Channel Connectivity change to CONNECTING
```

Proof

# xDS

- X Discovery Service
- gRPC Proxyless Service Mesh, still experimental in Istio 1.11+
- Not recommended
- <https://istio.io/latest/blog/2021/proxyless-grpc/>

```

apiVersion: networking.istio.io/v1alpha3
kind: VirtualService
metadata:
  name: cmdb-server-vsvc
  namespace: app-titan
spec:
  hosts:
  - cmdb-server.app-titan.svc.cluster.local
  http:
  - name: primary
    timeout: 10s
    route:
    - destination:
        host: cmdb-server-stable
        subset: v1
        weight: 100
    - destination:
        host: cmdb-server-canary
        subset: v1
        weight: 0

```

```
if enableXDS {
    log.Println(v...: "using xds server")
    cred, err := xdscred.NewServerCredentials(xdscred.ServerOptions{
        FallbackCreds: insecure.NewCredentials(),
    })
    if err != nil {
        panic(err)
    }
    opts = append(opts, grpc.Creds(cred))
    grpcServer = xds.NewGRPCServer(opts...)
}
```

# xDS - Server side

xds.NewGRPCServer instead of grpc.NewServer

# xDS gRPC server deployment

```
template:
  metadata:
    annotations:
      sidecar.istio.io/inject: "true"
      inject.istio.io/templates: grpc-agent
  labels:
    app: advanced-grpc
    subset: server
```

Add annotations

```
/ # env | grep XDS
GRPC_XDS_BOOTSTRAP=/var/lib/istio/data/grpc-bootstrap.json
GRPC_XDS_EXPERIMENTAL_SECURITY_SUPPORT=true
```

Results to env var

# xDS Client side

```
xds:///advanced-grpc-server.app-nerds.svc.cluster.local:8023
```

## Add schema

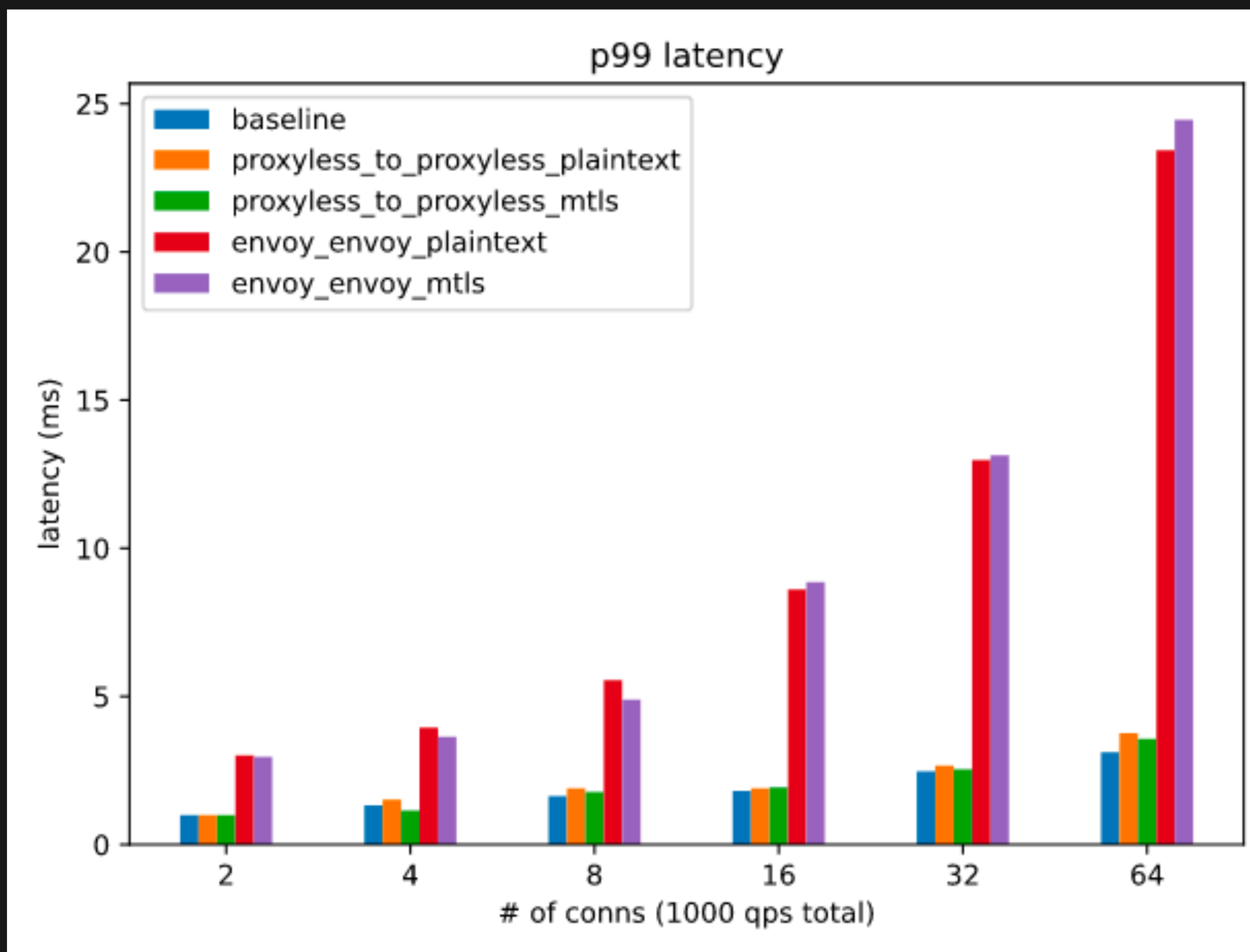
```
template:
  metadata:
    annotations:
      sidecar.istio.io/inject: "true"
      inject.istio.io/templates: grpc-agent
    labels:
      app: advanced-grpc
      subset: server
```

## Add annotations

| Name                                                         | State | Channel | SubChannel |
|--------------------------------------------------------------|-------|---------|------------|
| xds:///advanced-grpc-server.app-nerds.svc.cluster.local:8023 | IDLE  | 0       | 0          |
| unix:///etc/istio/proxy/XDS                                  | READY | 0       | 1          |
| 127.0.0.1:33695                                              | READY | 0       | 1          |

## Proof

# xDS Benchmark



# Connection pooling

- Istio + Envoy: dead simple way
- [go.planetmeican.com/bazaar/rpc.Pool](https://go.planetmeican.com/bazaar/rpc.Pool): flawed
- Resolver + LBPolicy: elegant but hardcore

```
/ # client grpc unary --endpoint advanced-grpc-server.app-nerds.svc.cluster.local:8023 --number 999
```

Launch 999 requests

| ID | Name   | LocalAddr | Calls | Success | Fail | LastCall |
|----|--------|-----------|-------|---------|------|----------|
| 1  | <none> | <none>    | 0     | 0       | 0    | none     |
| 2  | <none> | :::8023   | 336   | 335     | 0    | 109ms    |

| ID | Name   | LocalAddr | Calls | Success | Fail | LastCall |
|----|--------|-----------|-------|---------|------|----------|
| 1  | <none> | <none>    | 0     | 0       | 0    | none     |
| 2  | <none> | :::8023   | 334   | 333     | 0    | 111ms    |

| ID | Name   | LocalAddr | Calls | Success | Fail | LastCall |
|----|--------|-----------|-------|---------|------|----------|
| 1  | <none> | <none>    | 0     | 0       | 0    | none     |
| 2  | <none> | :::8023   | 334   | 333     | 0    | 109ms    |

3 server stats

# Connection Pooling

## Istio Envoy

# Connection Pooling

[go.planetmeican.com/  
bazaar/rpc.Pool](https://go.planetmeican.com/bazaar/rpc.Pool)

```
pool, err := rpc.New(func() (*grpc.ClientConn, error) {
    ctx, cancel := context.WithTimeout(context.TODO(), c.DialTimeout)
    defer cancel()

    opts := []grpc.DialOption{
        grpc.WithTransportCredentials(insecure.NewCredentials()),
        grpc.WithBlock(),
        grpc.WithChainUnaryInterceptor(grpcmw.TelemetryClientUnaryInterceptor(tele)),
    }
    cc, err := grpc.DialContext(ctx, c.Endpoint, opts...)
    if err != nil {
        return nil, err
    }
    return cc, nil
}, c.PoolSize, c.PoolSize, time.Minute)
```

Create

```
cc, err := b.pool.Get(ctx)
if err != nil {
    log.Error(msg: "pool.Get failed", log.ErrField(err))
    return
}
defer cc.Close()

pb.NewPocClient(cc).Ping(ctx, new(emptypb.Empty))
```

Use

# Connection Pooling

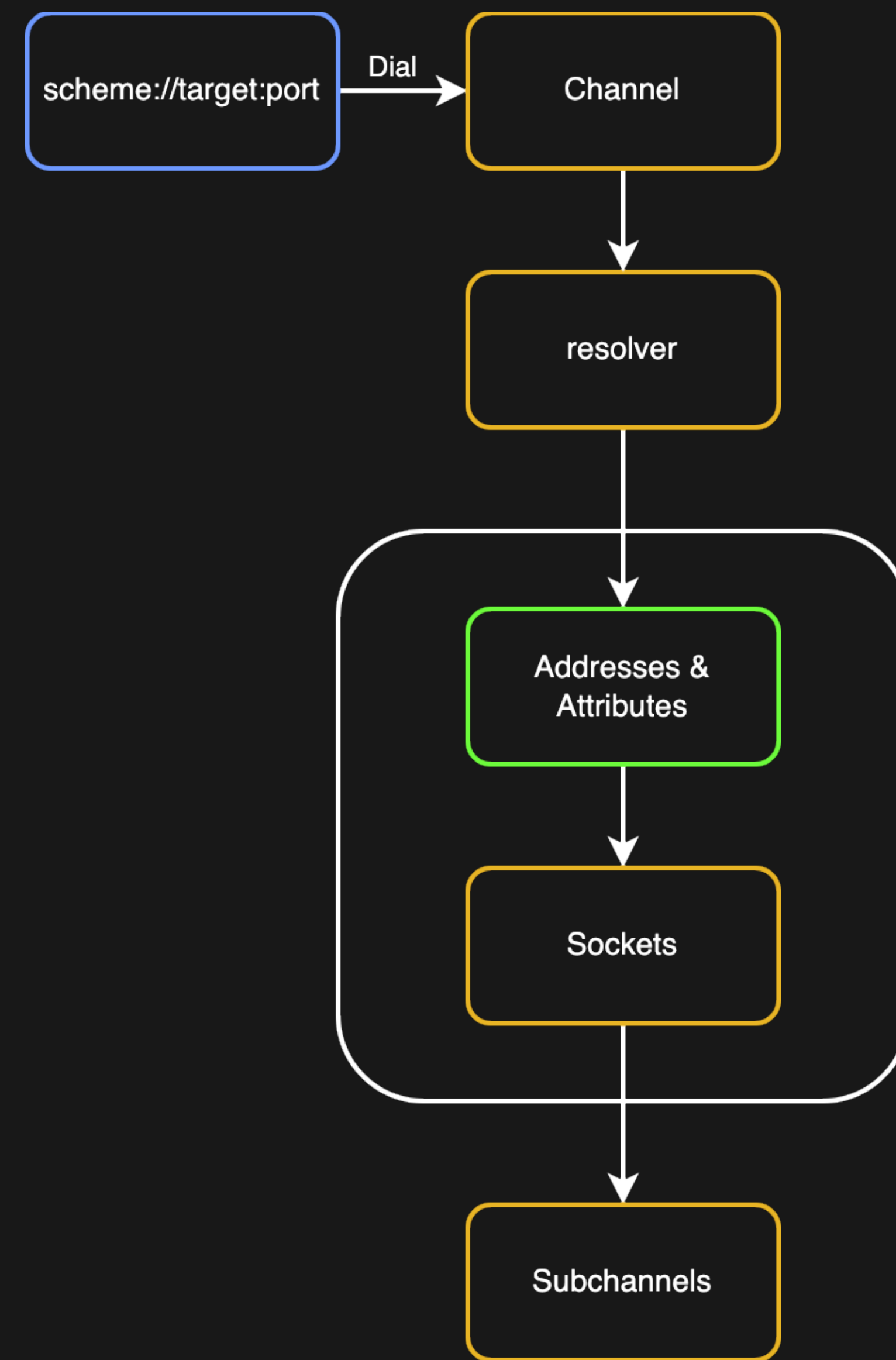
- [go.planetmeican.com/bazaar/rpc.Pool](http://go.planetmeican.com/bazaar/rpc.Pool)
- Flawed:
  - Too many channels
  - Mutex for each request

# Connection Pooling

## Resolver + LB Policy

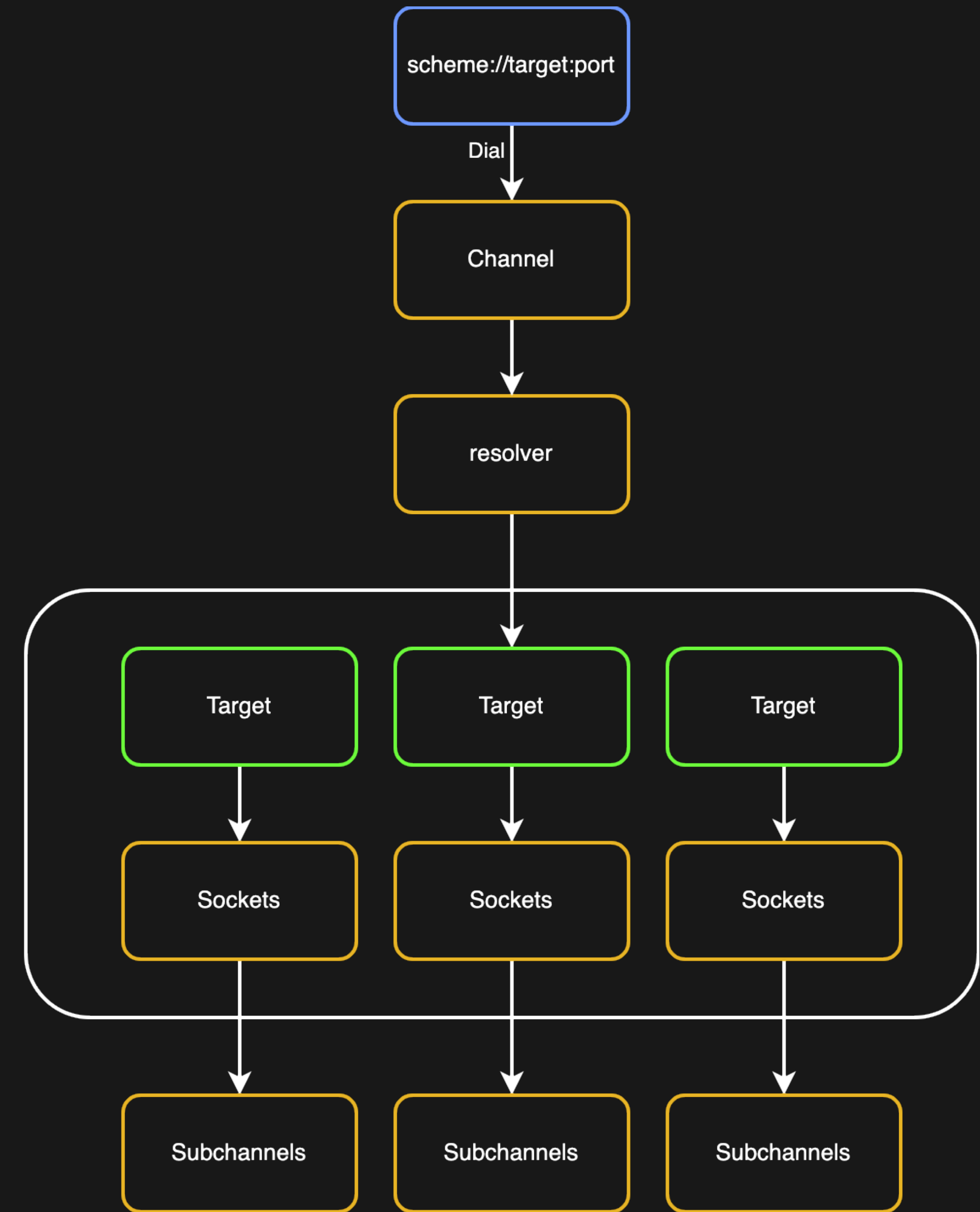
# What is a gRPC resolver ?

- endpoint -> scheme -> addresses
- Endpoint:
  - target:port
  - scheme:///target:port
- scheme -> resolver
- <https://github.com/grpc/grpc/blob/master/doc/naming.md>



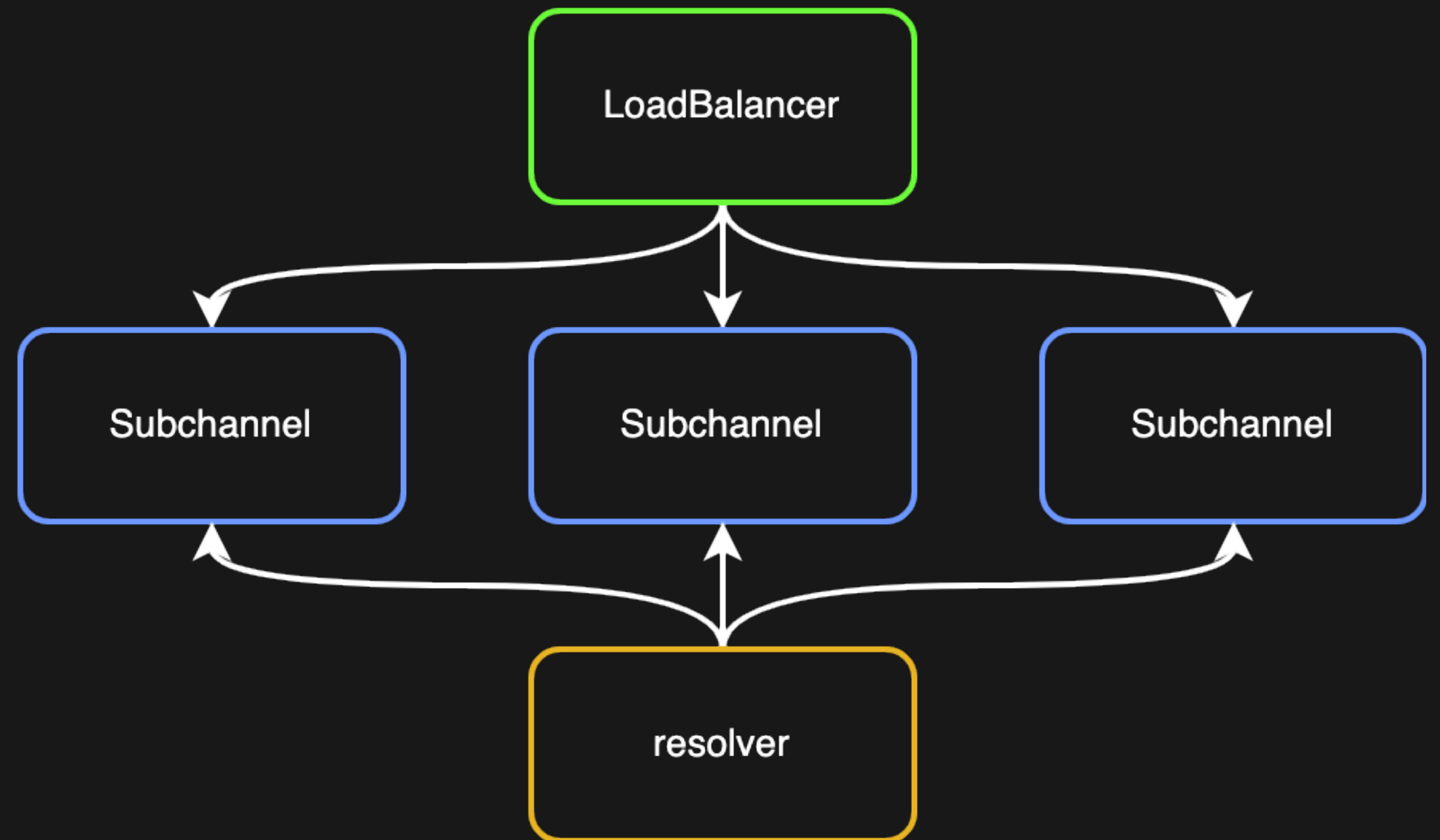
# gRPC Resolver

## Resolve multiple targets



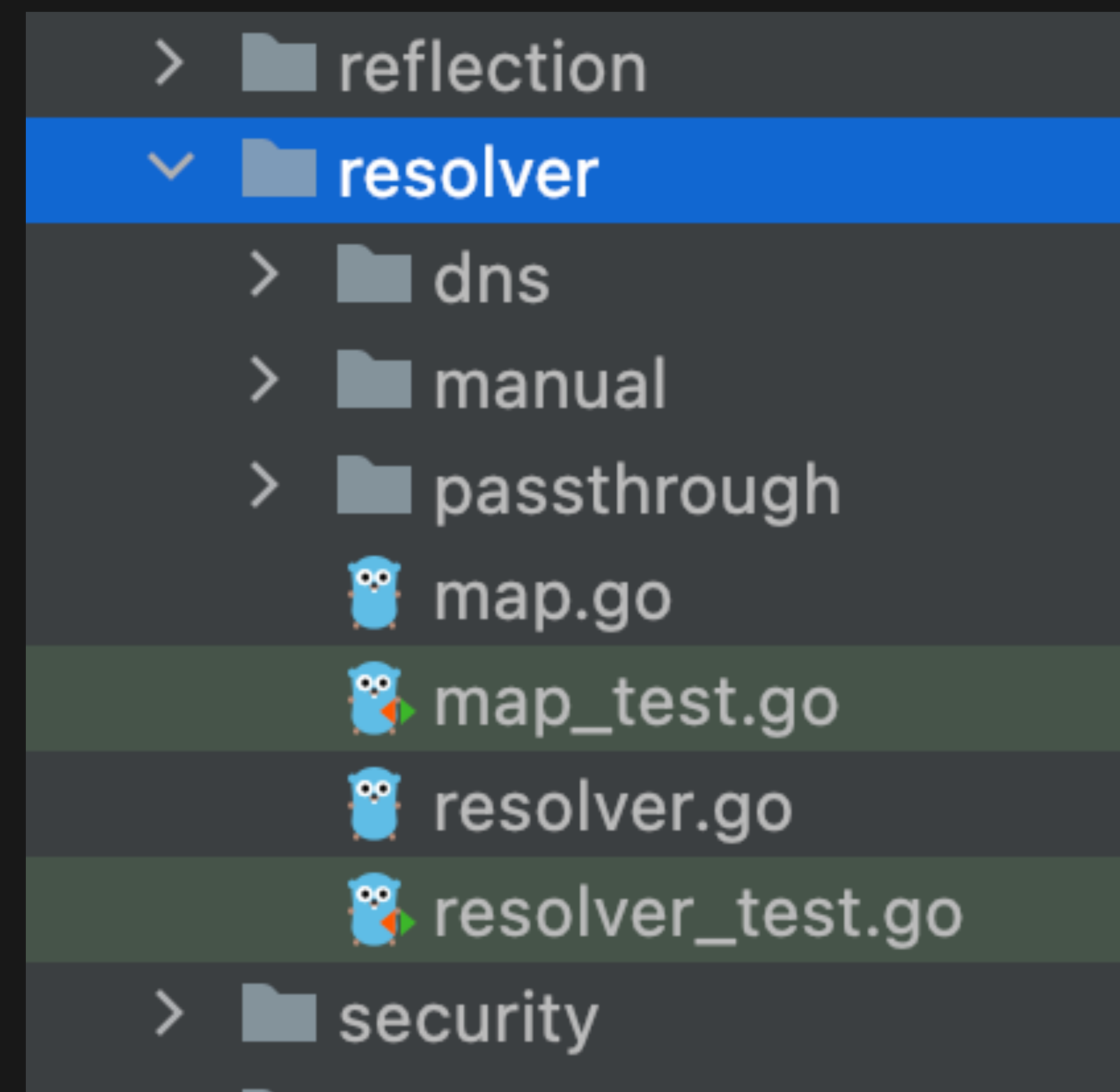
# Resolver + LoadBalancer

- Get together
- Resolver -> multiple subchannels
- LoadBalancer -> balance between subchannels



# gRPC Resolvers

- Resolvers:
  - passthrough (default)
  - dns (dns targets)
  - manual (user implementation)
  - xds (external system)
- Source: [google.golang.org/grpc/resolver](https://google.golang.org/grpc/resolver)



# Passthrough Resolver

- Pass request to DNS server
- DNS server returns IPs
- advanced-grpc-server.app-nerds.svc.cluster.local  
-> 10.100.11.219

```
$ kubectl -n app-nerds get svc advanced-grpc-server
```

| NAME                 | TYPE      | CLUSTER-IP   | EXTERNAL-IP | PORT(S)           | AGE  |
|----------------------|-----------|--------------|-------------|-------------------|------|
| advanced-grpc-server | ClusterIP | 10.100.70.12 | <none>      | 8023/TCP,8024/TCP | 5d1h |

# DNS Resolver

- DNS resolver:
  - "dns:///target:port"
- Resolve DNS first
- ClusterIP service:
  - advanced-grpc-server.app-nerds.svc.cluster.local  
-> 10.100.11.219

```
$ kubectl -n app-nerds get svc advanced-grpc-server
```

| NAME                 | TYPE      | CLUSTER-IP   | EXTERNAL-IP | PORT(S)           | AGE  |
|----------------------|-----------|--------------|-------------|-------------------|------|
| advanced-grpc-server | ClusterIP | 10.100.70.12 | <none>      | 8023/TCP,8024/TCP | 5d1h |

# DNS Resolver

- Load balance prerequisites: headless service
  - advanced-grpc-server-hs.app-nerds.svc.cluster.local ->
    - 172.24.132.140
    - 172.24.147.240
    - 172.24.164.129
- Results to multiple subchannels

```
$ kubectl -n app-nerds describe endpoints advanced-grpc-server-hs
Name:          advanced-grpc-server-hs
Namespace:    app-nerds
Labels:       app=advanced-grpc
              service.kubernetes.io/headless=
              stage=headless
              subset=server
Annotations:  endpoints.kubernetes.io/last-change-trigger-time: 2022-12-31T05:58:52Z
Subsets:
  Addresses:          172.24.132.140,172.24.147.240,172.24.164.129
  NotReadyAddresses:  <none>
  Ports:
    Name      Port  Protocol
    ----      -
    http-grpcgw 8024  TCP
    grpc       8023  TCP
```

```
/ # client grpc unary --endpoint dns:///advanced-grpc-server-hs.app-nerds.svc.cluster.local:8023 --load-balance --print-channelz
```

endpoint: dns:///...

| Name                                                            | State | Channel | SubChannel | Calls | Success | Fail | LastCall |
|-----------------------------------------------------------------|-------|---------|------------|-------|---------|------|----------|
| dns:///advanced-grpc-server-hs.app-nerds.svc.cluster.local:8023 | READY | 0       | 3          | 1     | 1       | 0    | 25ms     |
| 127.0.0.1:44325                                                 | READY | 0       | 1          | 1     | 0       | 0    | 0ms      |

subchannel \* 3

```
2022-12-31 06:07:34.573899082 +0000 UTC
Channel switches to new LB policy "round_robin"
Subchannel(id:4) created
Subchannel(id:5) created
Subchannel(id:6) created
Channel Connectivity change to CONNECTING
Channel Connectivity change to READY
```

subchannels + LB Policy

DNS Resolver

+ Headless service

# Manual Resolver

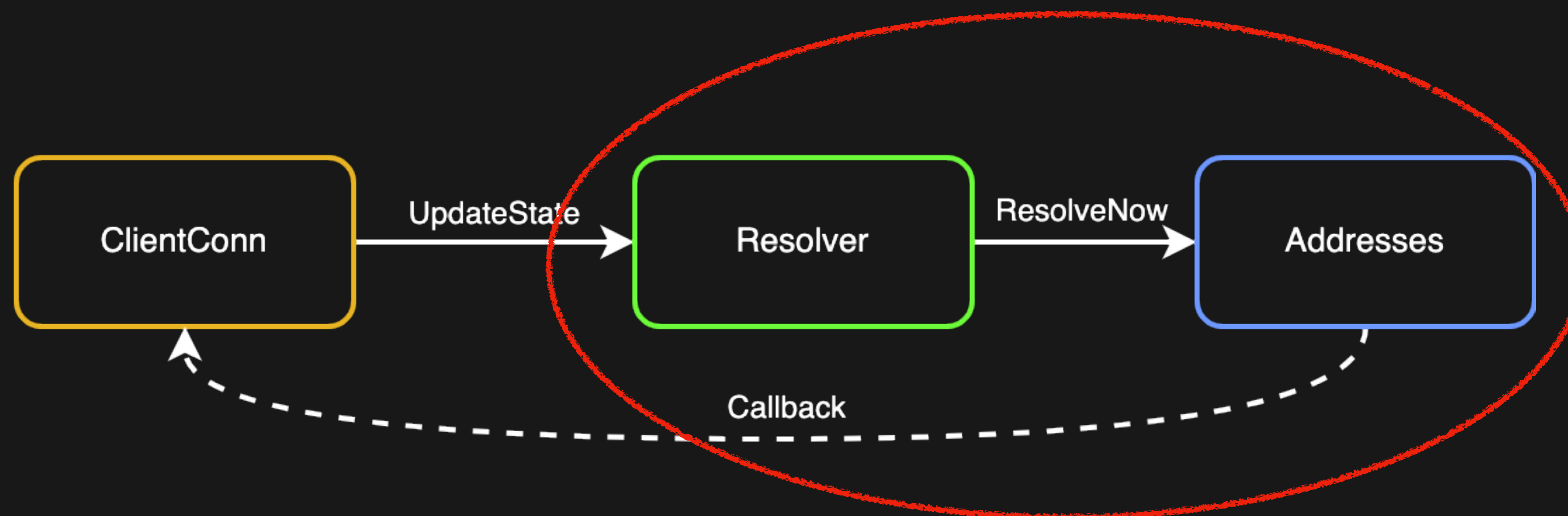
## “Hardcore”

```
// Resolver is also a resolver builder.
// It's build() function always returns itself.
type Resolver struct {
    // BuildCallback is called when the Build method is called. Must not be
    // nil. Must not be changed after the resolver may be built.
    BuildCallback func(resolver.Target, resolver.ClientConn, resolver.BuildOptions)
    // ResolveNowCallback is called when the ResolveNow method is called on the
    // resolver. Must not be nil. Must not be changed after the resolver may
    // be built.
    ResolveNowCallback func(resolver.ResolveNowOptions)
    // CloseCallback is called when the Close method is called. Must not be
    // nil. Must not be changed after the resolver may be built.
    CloseCallback func()
    scheme string

    // Fields actually belong to the resolver.
    CC resolver.ClientConn
    bootstrapState *resolver.State
}
```

# Manual Resolver

- Tricks: One target -> multiple addresses -> multiple subchannels for LB



```

if poolSize > 1 {
    var (
        rb resolver.Builder
        err error
    )
    endpoint, rb, err = poolresolver.GetPoolResolver(endpoint, poolSize)
    if err != nil {
        return nil, fmt.Errorf(format: "poolresolver.GetPoolResolver for %q failed: %w", endpoint, err)
    }
    dopts = append(dopts, grpc.WithResolvers(rb))
}
return grpc.DialContext(context.TODO(), endpoint, dopts...)

```

# Manual Resolver

[go.planetmeican.com/nerds/app/v2/components/grpc/client/poolresolver](https://go.planetmeican.com/nerds/app/v2/components/grpc/client/poolresolver)

```
/ # client grpc unary \  
  --endpoint advanced-grpc-server-hs.app-nerds.svc.cluster.local:8023 \  
  --load-balance \  
  --print-channelz \  
  --pool-size 10
```

Pool size = 10

| Name                                                                      | State | Channel | SubChannel | Calls | Success | Fail | LastCall |
|---------------------------------------------------------------------------|-------|---------|------------|-------|---------|------|----------|
| passthrough+pool:///advanced-grpc-server.app-nerds.svc.cluster.local:8023 | READY | 1       | 10         | 1     | 1       | 0    | 47ms     |
| 127.0.0.1:37491                                                           | READY | 0       | 1          | 1     | 0       | 0    | 0ms      |

scheme changed

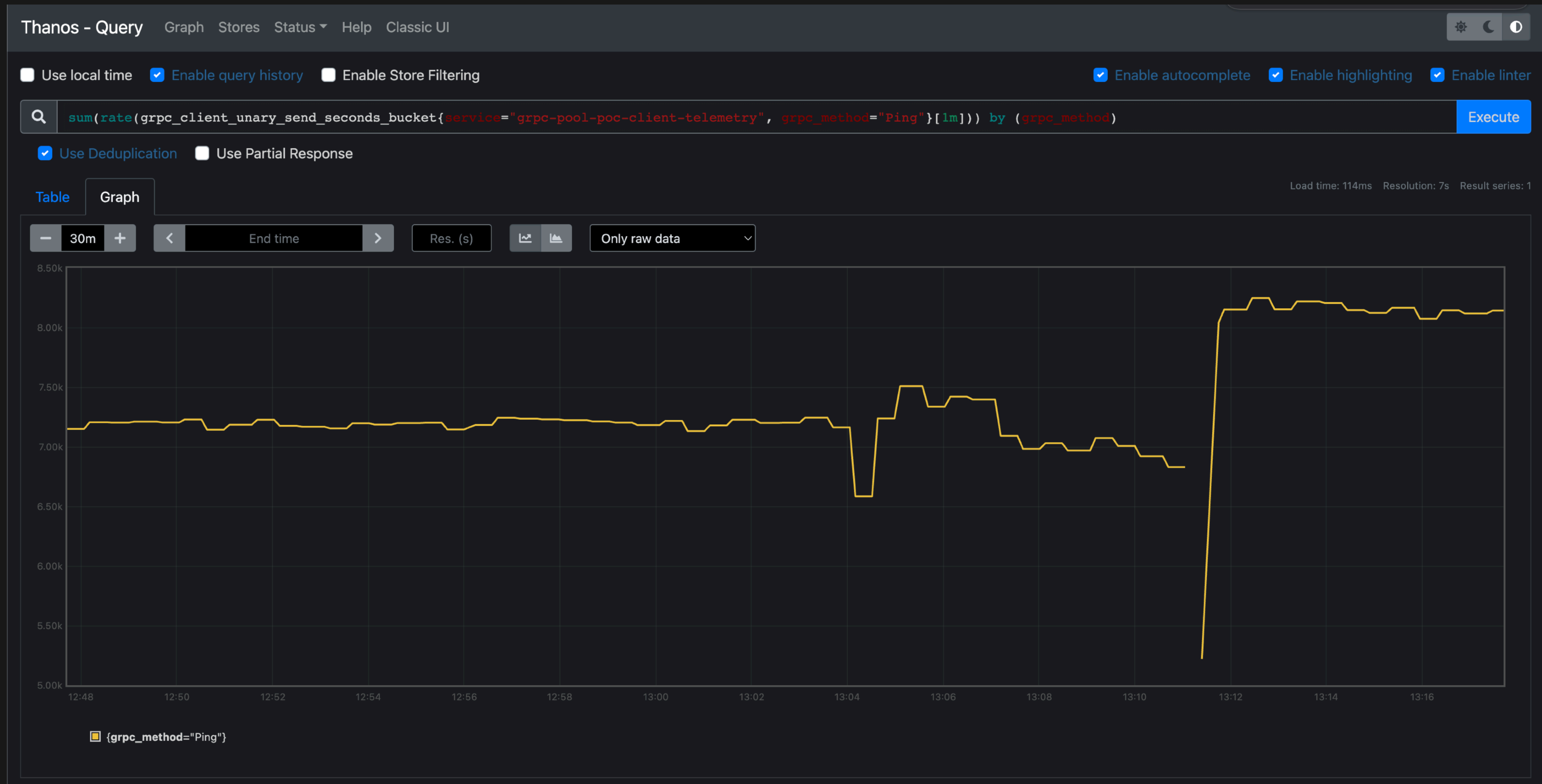
# Manual Resolver

1 Channel, 10 subchannels

# Connection Pooling Benchmark



- 2500 Concurrency
- 5 servers
- 1 client:
  - 1.5 vCPU
  - 500Mi Memory



# Connection Pooling Benchmark

## Istio (7k) vs Manual Resolver (8k) - QPS



# Connection Pooling Benchmarking nerds/bazaar/rpc.Pool





# Connection Pooling Benchmark

Resources consumption

# Chaos Testing w/ Istio

| Channels: <none> |      |       |       |           |        |
|------------------|------|-------|-------|-----------|--------|
| Subchannels:     |      |       |       |           |        |
| ID               | Name | State | Start | Succeeded | Failed |
| 2                |      | READY | 10000 | 9998      | 2      |
| Trace:           |      |       |       |           |        |

w/o pooling

| Subchannels: |      |       |       |           |        |
|--------------|------|-------|-------|-----------|--------|
| ID           | Name | State | Start | Succeeded | Failed |
| 8            |      | READY | 1000  | 999       | 1      |
| 7            |      | READY | 1001  | 1000      | 1      |
| 9            |      | READY | 1000  | 1000      | 0      |
| 10           |      | READY | 1000  | 1000      | 0      |
| 2            |      | READY | 1000  | 1000      | 0      |
| 3            |      | READY | 1000  | 1000      | 0      |
| 4            |      | READY | 1001  | 1001      | 0      |
| 5            |      | READY | 999   | 999       | 0      |
| 6            |      | READY | 999   | 999       | 0      |
| 11           |      | READY | 1000  | 999       | 1      |
| Trace:       |      |       |       |           |        |

w/ pooling

- Timeout 2s
- Rolling update pods
- 10000 requests

# Chaos Testing w/ Istio

- Timeout 2s -> 5s
- Rolling update pods
- 10000 requests
- All good

# Chaos Testing w/o Istio

- Timeout 2s
- Rolling update pods
- 10000 requests
- No error (not always)

| Name                                                  | State | Channel | SubChannel | Calls | Success | Fail | LastCall |
|-------------------------------------------------------|-------|---------|------------|-------|---------|------|----------|
| advanced-grpc-server.app-nerds.svc.cluster.local:8023 | READY | 0       | 1          | 10000 | 10000   | 0    | 0ms      |
| 127.0.0.1:36315                                       | READY | 0       | 1          | 1     | 0       | 0    | 0ms      |

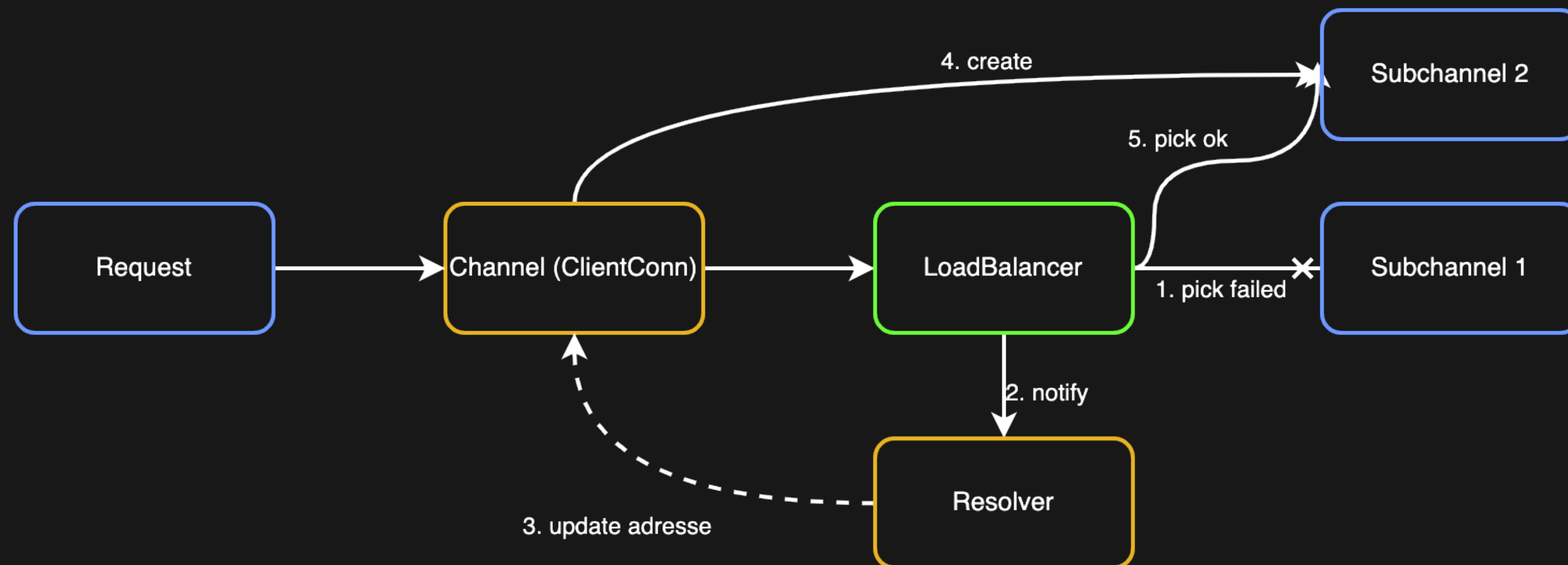
w/o pooling

| Name                                                                      | State | Channel | SubChannel | Calls | Success |
|---------------------------------------------------------------------------|-------|---------|------------|-------|---------|
| passthrough+pool:///advanced-grpc-server.app-nerds.svc.cluster.local:8023 | READY | 0       | 10         | 10000 | 10000   |
| 127.0.0.1:40263                                                           | READY | 0       | 1          | 1     | 0       |

w/ pooling

# Why failed calls with Istio ?

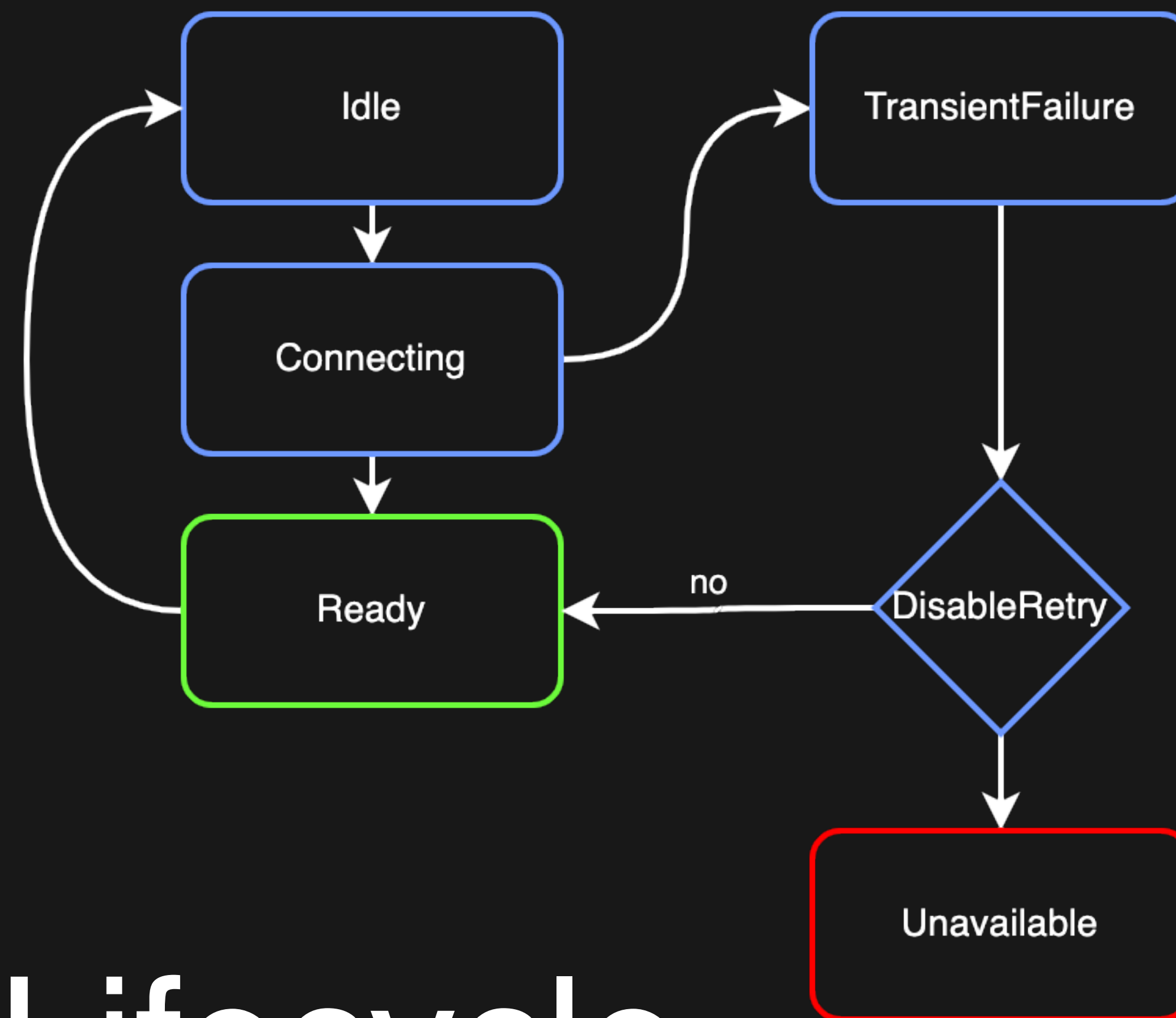
(Not always though)



# ClientConn Lifecycle

1 channel - 1 subchannel

```
const (
    // Idle indicates the ClientConn is idle.
    Idle State = iota
    // Connecting indicates the ClientConn is connecting.
    Connecting
    // Ready indicates the ClientConn is ready for work.
    Ready
    // TransientFailure indicates the ClientConn has seen a failure but expects to recover.
    TransientFailure
    // Shutdown indicates the ClientConn has started shutting down.
    Shutdown
)
```



# ClientConn Lifecycle

State flow

| Calls | Success | Fail |
|-------|---------|------|
| 10000 | 5284    | 4716 |
| 1     | 0       | 0    |

# ClientConn Lifecycle

## Disable retry

```
Channel switches to new LB policy "round_robin"
Subchannel(id:2) created
Channel Connectivity change to CONNECTING
Channel Connectivity change to READY
Channel Connectivity change to IDLE
Channel Connectivity change to CONNECTING
Channel Connectivity change to READY
Channel Connectivity change to IDLE
Channel Connectivity change to CONNECTING
Channel Connectivity change to TRANSIENT_FAILURE
Channel Connectivity change to READY
```

**Transient failure results to failed invocation**

```
2023/01/01 13:33:40 request 8846 failed:
  rpc error: code = Unavailable desc = last connection error:
    connection error: desc =
      "transport: Error while dialing dial tcp 10.100.70.12:8023: connect: connection refused"
```

# ClientConn Lifecycle

## Enable retry (by default)

| Calls | Success | Fail |
|-------|---------|------|
| 10000 | 10000   | 0    |
| 1     | 0       | 0    |

```
2022-12-31 08:23:12.569237731 +0000 UTC
Channel switches to new LB policy "round_robin"
Subchannel(id:2) created
Channel Connectivity change to CONNECTING
Channel Connectivity change to READY
Channel Connectivity change to IDLE
Channel Connectivity change to CONNECTING
Channel Connectivity change to READY
Channel Connectivity change to IDLE
Channel Connectivity change to CONNECTING
Channel Connectivity change to READY
Channel Connectivity change to IDLE
Channel Connectivity change to CONNECTING
Channel Connectivity change to READY
```

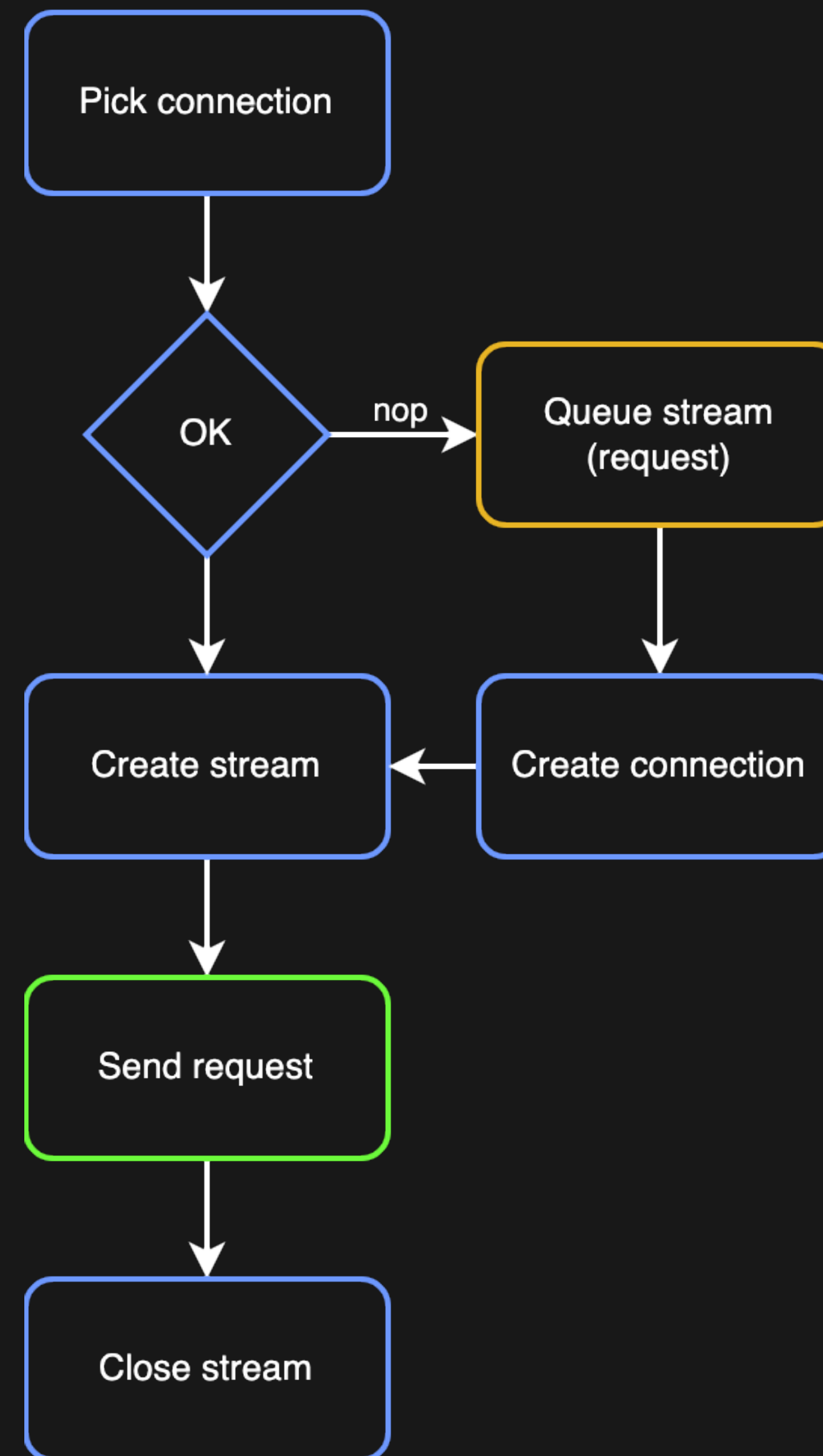
Reconnect instead of returning directly error to the client

# Envoy

- Connection Pooling
- Standard case

```

envoy pool      queueing stream due to no available connections
envoy pool      trying to create new connection
envoy pool      creating a new connection
envoy http2     [C3914] updating connection-level initial window size to 268435456
envoy connection [C3914] connecting to 172.24.145.33:8023
envoy connection [C3914] connection in progress
envoy connection [C3914] connected
envoy pool      [C3914] attaching to next stream
envoy pool      [C3914] creating stream
envoy pool      [C3914] destroying stream: 0 remaining
envoy http2     [C3914] stream closed: 0
  
```

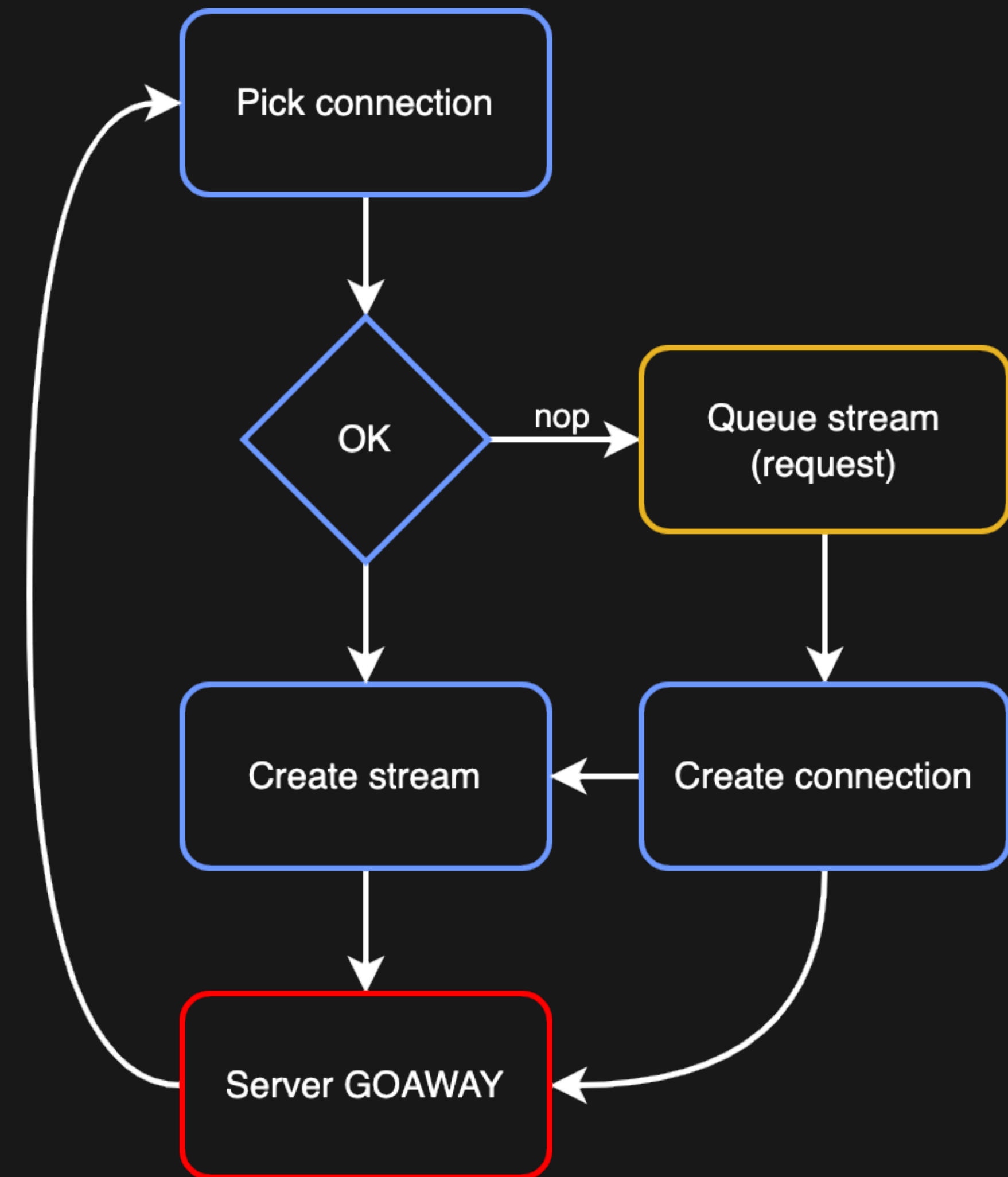


# Envoy

- Server shutdown while sending request
- May timeout

```

envoy pool queueing stream due to no available connections
envoy pool trying to create new connection
envoy pool creating a new connection
envoy http2 [C3904] updating connection-level initial window size to 268435456
envoy connection [C3904] connecting to 172.24.144.168:8023
envoy connection [C3904] connection in progress
envoy connection [C3904] connected
envoy pool [C3904] attaching to next stream
envoy pool [C3904] creating stream
envoy pool [C3904] remote goaway
envoy pool [C3904] destroying stream: 0 remaining
envoy connection [C3904] closing data_to_write=0 type=1
envoy connection [C3904] closing socket: 1
envoy connection [C3904] SSL shutdown: rc=0
envoy pool [C3904] client disconnected, failure reason:
envoy pool queueing stream due to no available connections
envoy pool trying to create new connection
envoy pool creating a new connection
  
```

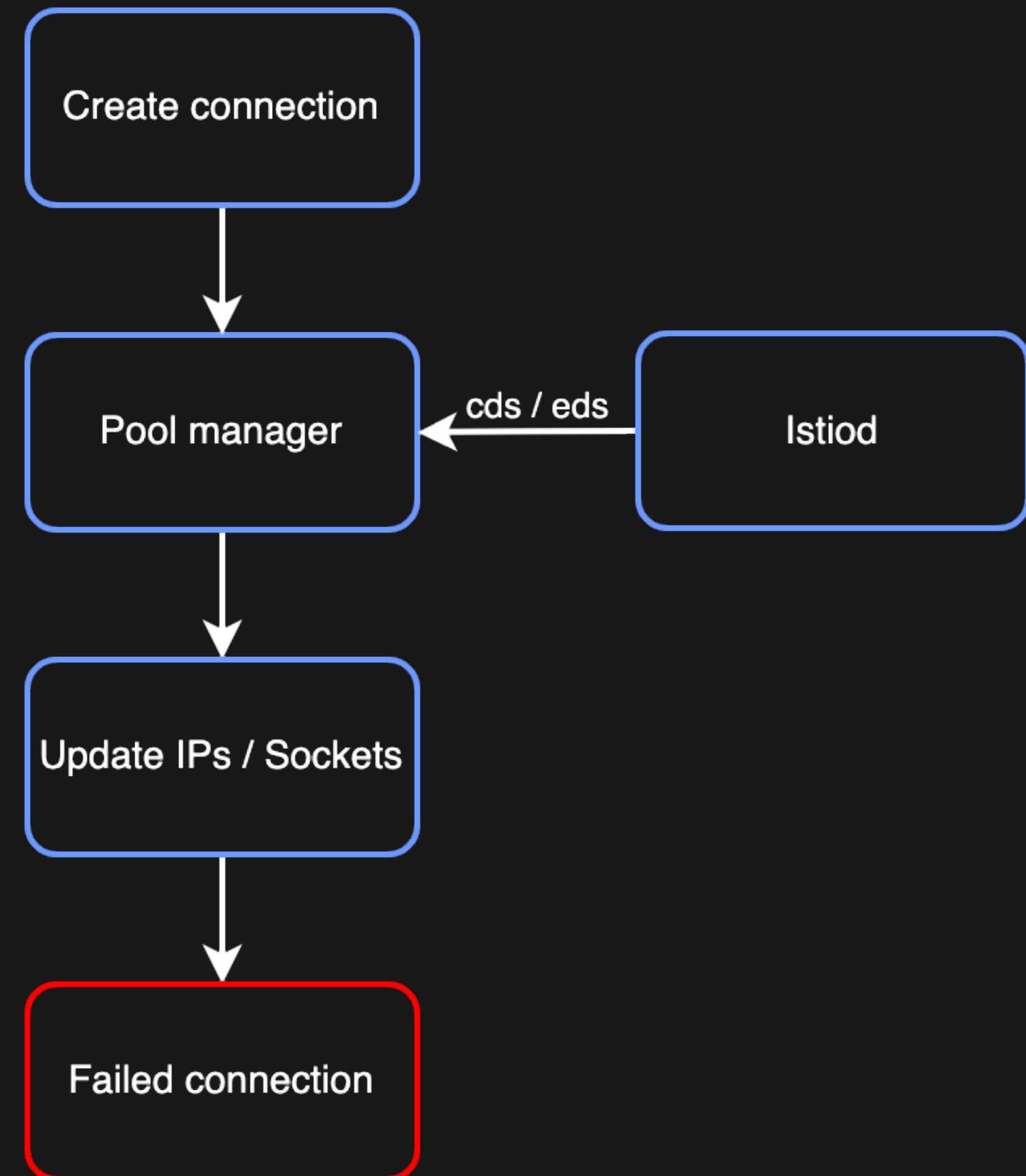


# Envoy

- Slow CDS / EDS results to incorrect endpoint list

```

envoy pool      queueing stream due to no available connections
envoy pool      trying to create new connection
envoy pool      creating a new connection
envoy http2     [C1834] updating connection-level initial window size to 268435456
envoy client    [C1834] connecting
envoy connection [C1834] connecting to 172.24.169.139:8023
envoy connection [C1834] connection in progress
envoy http      [C1594][S8689401928084442054] request end stream
envoy connection [C1834] delayed connection error: 111
envoy connection [C1834] closing socket: 0
envoy client    [C1834] disconnect. resetting 0 pending requests
envoy pool      [C1834] client disconnected, failure reason:
envoy router    [C1594][S8689401928084442054] upstream reset: reset reason: connection failure,
envoy conn_handler [C1835] new connection from 172.24.148.140:40390
envoy http      [C1835] new stream
  
```



# Preventing request failure with Istio

- Increase timeout at gRPC level
- Add Envoy retry policy
  - Analysis
- Pitfalls
  - It overrides request timeout at gRPC level

```

apiVersion: networking.istio.io/v1alpha3
kind: VirtualService
metadata:
  name: advanced-grpc-server-vs
  namespace: app-nerds
spec:
  hosts:
  - advanced-grpc-server.app-nerds.svc.cluster.local
  http:
  - name: primary
    timeout: 10s
    retries:
      attempts: 3
      perTryTimeout: 5s
      retryOn: unavailable
    route:
      destination:
        host: advanced-grpc-server
        subset: v1
        weight: 100

```

# Preventing request failure with Istio



- Rolling update
- Random pod kill
- 100,000 requests
- No error

# Conclusion

- Load balancing
  - round\_robin
- Connection Pooling
  - Simple way: no pooling, use Istio
  - Hard way: manual resolver
- Analyze on Istio pitfalls
  - Add retry, or embrace the error rate

# What's next ?

- Streaming
- Healthz
- Public internet
- Keep alive
- Dynamic message
- Extend proto-gen

# References

- <https://grpc.io/blog/wireshark/>
- <https://halfrost.com/http2-http-frames-definitions/#toc-12>
- <https://github.com/grpc/grpc/blob/master/doc/load-balancing.md>

Fin.

Questions ?

```
import "google.golang.org/grpc/health/grpc_health_v1"
```

## Import

```
grpc_health_v1.RegisterHealthServer(grpcServer, healthz.NewServer())
```

## Register

```
651844176281.dkr.ecr.cn-northwest-1.amazonaws.com.cn/titan/base-images/alpine-tz8-grpc:v3.16.0
```

## Base docker image

```
{"status": "SERVING"}
```

## Response

```
readinessProbe:
  exec:
    command:
      - grpc_health_probe
      - '-addr=:8023'
  initialDelaySeconds: 8
  periodSeconds: 5
  successThreshold: 1
  timeoutSeconds: 5
  failureThreshold: 5
```

## Deployment

# Healthz

## Server health status